

FOUNDRY ARCHITECTURE FOR ENTERPRISE AI

Study Guide — Data Products, Ontology-First Design & Domain-Driven Modeling

Week 1 · Day 1 · Prepared by Rajesh Pasham

Learning Objectives

By the end of today, you will be able to:

- Explain what a Data Product is and why Foundry exposes data this way
- Explain why this program designs Ontology-first rather than pipeline-first
- Apply Domain-Driven Design principles to decide what becomes an Object Type
- Describe the anatomy of an Object Type: Properties, Shared Properties, Link Types, and Action Types
- Use branching to isolate in-progress Ontology changes from production
- Explain why incremental processing is preferred over full rebuilds

Program Context

Item	Detail
Session Type	Concepts and architecture lecture — no live Palantir platform work today; every example in this guide is presented in writing and diagrams
Session Format	One combined live session for India & Europe — 2:00–3:00 PM IST / 10:30–11:30 AM CEST
Platform	Delivered hands-on using individual Palantir Developer Tier accounts — fully browser-based, nothing to install
Shared Domain, Distinct Use Cases	All batches work in Insurance. This guide's Customer/Policy/Claim example is the shared reference model — each batch will apply the same method to its own assigned use case (Claims Operations, Underwriting, Fraud & SIU, or Policy Servicing) starting Day 3
Use Case Finalization	The use case allocation referenced in this program is the trainer's starting point — the exact use case per batch is confirmed with participants during Week 1

1. Why This Module Matters

Every later week in this program depends on what gets built today. A Copilot with no Ontology has nothing to reason over. An Action has nothing to write to without an Object Type. A multi-agent workflow has no relationships to navigate without Link Types. Today is not "setup" — it is the architectural decision that determines how well everything else works.

2. Key Concepts

Concept	Definition
Data Product	A governed dataset or model exposed through standard output ports — an API and/or the Ontology — so it can be discovered and consumed without exposing how it was built.
Ontology	The operational layer and digital twin of the organization: the objects, properties, and relationships that represent real-world business concepts, plus the actions that can change them.
Object Type	The definition of an entity or event in the business — Customer, Policy, Claim.
Property	An attribute of an Object Type — claimAmount, fraudScore.
Link Type	The schema definition of a relationship between two Object Types.
Action Type	The schema definition of a set of changes an Object Type can undergo, including side effects.
Branch	An isolated working copy of the Ontology used to model changes safely before merging.

3. Data Products: The Foundation

A Data Product is a governed dataset or model, exposed through standard output ports so that other teams can discover and consume it without needing to understand — or depend on — how it was built internally.

- A producing team builds and owns the dataset or model (for example, Claims)
- The product is exposed through an API and/or the Ontology, not a raw table
- It is discoverable through Data Catalog or Ontology Manager, not informal channels
- Consumers build on top of the product without touching its internals

This matters because it is what makes an enterprise Ontology sustainable at scale: teams can evolve their own pipelines independently as long as they keep their Data Product's contract stable.

4. Why Ontology-First?

The traditional approach builds ETL pipelines first and lets the data model emerge later — which means every downstream consumer ends up re-deriving their own definition of "a claim" or "a customer," and those definitions drift apart over time.

This program designs Ontology-first: agree on what a Claim, Policy, and Customer are — once, with the business — and then build pipelines that feed that shared model. The Ontology sits on top of your data and models and connects them to their real-world counterparts, so every application, Copilot, and Agent built in later weeks reasons against the same shared definitions.

5. Domain-Driven Ontology Modeling

Object Type design benefits from borrowing directly from Domain-Driven Design (DDD):

- Identify the Bounded Context — Claims processing and Underwriting may use the same word ("risk") differently; model them as separate contexts rather than forcing one shared definition
- Capture the Ubiquitous Language — use the exact terms domain experts use, not the column names in a source database

- Map Language to Object Types — "a claim has a policy, a claimant, and a status" becomes the Claim, Policy, and Customer Object Types, with status as a property
- Validate with Domain Experts — an adjuster or underwriter should recognize the resulting Ontology as how they actually talk about their work

6. Anatomy of an Object Type

Element	Description
Properties	Attributes specific to this Object Type, e.g. claimAmount, fraudScore, documentsComplete
Shared Properties	Centrally managed properties reused consistently across multiple Object Types, e.g. createdAt, lastModifiedBy
Link Types	The relationship schema between Object Types, e.g. Claim links to Policy, Policy links to Customer
Action Types	The schema for how an Object Type can be changed — parameters, submission criteria, and side effects

Shared properties are worth calling out specifically: centralizing common attributes (like audit fields) avoids each team inventing a slightly different version of the same concept, which is one of the most common sources of Ontology drift.

Adopt: model from interviews with domain experts; centralize common attributes as Shared Properties; define cardinality from what's explicitly stated, not assumed.

Avoid: modeling straight from a source database schema; letting every Object Type invent its own audit fields; guessing at relationships "to save time."

7. Branching Strategies

Pattern	When to Use It
Feature Branch	Model a new Object Type or property change in isolation, then merge once validated
Environment Branch	Separate development and staging Ontology states before promoting to production
Short-Lived by Design	Branches should be merged quickly — long-lived branches drift and become painful to reconcile

8. Incremental Processing

A full rebuild reprocesses the entire dataset on every run: simple to reason about, but slow and expensive as data volume grows. Incremental processing updates the Ontology from only new or changed records since the last run — the standard pattern for production-scale pipelines, and the pattern this program uses throughout.

Approach	When It's the Right Call
Full Rebuild	Small data volume, logic still changing frequently during early development, or correctness matters more than speed
Incremental Processing	Large data volume, a pipeline running on a schedule, or freshness at production scale is the goal

This program treats incremental processing as the standard pattern from Week 1 onward, matching production practice — full rebuilds are the exception, not the default.

9. Architecture Walkthrough

Today's concepts compose into one architecture:

- Source Systems — Claims, policy, and customer systems generate raw data
- Data Products — governed datasets/models exposed via API and Ontology output ports
- Ontology — Object Types, Link Types, and Action Types, modeled domain-first
- Branches — isolated space to evolve the model before merging to production
- Enterprise Consumers — the Copilots, Agents, and applications built in later weeks

10. Common Pitfalls

- Modeling straight from a source database schema instead of the business's actual language
- Skipping shared properties and letting every Object Type define its own version of common fields
- Leaving feature branches open for weeks, causing painful merge conflicts
- Defaulting to full rebuilds "because it's simpler," and only discovering the cost at production scale

11. Glossary

Term	Meaning
Data Product	A governed, discoverable dataset or model exposed via standard output ports
Ontology	The governed, operational model of business objects and their relationships
Object Type	The definition of a business entity or event
Link Type	The schema of a relationship between two Object Types
Action Type	The schema of a governed change to an Object Type
Branch	An isolated working copy of the Ontology
Bounded Context	A domain-modeling boundary within which a term has one consistent meaning

12. Further Practice

Continue with the accompanying hands-on exercise, "Model the Insurance Domain," where you will identify bounded contexts, define Object Types for Customer, Policy, and Claim, connect them with Link Types, and practice making a modeling change on a branch.